# 부록 A2 · 위성지도 GCS 실습 (1~4단)

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-10-29

**★ 이번 학기 녹화 강의 영상은 여기에 올라감 — 수업에서 놓친 부분은 영상으로 확인할 것**

**https://youtube.com/playlist?list=PLK_f1-krzJG8**

- 매주 수업 뒤 그 주차 영상이 이 재생목록에 추가됨
- 아래 "참조 강의" 는 **지난 학기까지의 선수 과목**이고, 이 재생목록은 **이번 학기 본 수업** 녹화임

**★ 참조 강의 — 본 과목이 전제하는 배경**

아래 다섯 과목은 **본 과목 담당 교수가 직접 강의한 것**이며, 본 과목이 전제하는 배경 지식에 해당함. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 됨. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것

| # | 과목 | 수준 | 언어 | 영상 | 자료·코드 |
|---|---|---|---|---|---|
| 1 | **제어공학** — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대 | 학부 | 한국어 | 재생목록 | 드라이브 |
| 2 | **제어시스템설계** — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현 | 학부 | 한국어 | 재생목록 | 드라이브 |
| 3 | **캡스톤디자인** — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지 | 학부 | 한국어 | 재생목록 | 드라이브 |
| 4 | **제어공학특론** — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림 | 대학원 | 영어 | 재생목록 | 드라이브 |
| 5 | **센서신호처리 및 융합** — 센서 모델, 잡음, 추정, 다중센서 융합 | 대학원 | 영어 | 재생목록 | 드라이브 |

**MATLAB·Simulink 가 처음이라면 4주차 실습 전에 아래를 끝낼 것.** 본 과목의 제어기 실습은 전부 Simulink 로 진행함. Onramp 는 무료이며 각각 몇 시간이면 끝남

| 도구 | 시작 지점 |
|---|---|
| MATLAB | MATLAB Onramp · Core MATLAB Skills |
| Simulink | Simulink Onramp · 담당 교수 Simulink 강의 1부 · 2부 |

### ★ 강의자료 저장소 — 처음 한 번만 `git clone`, 그 뒤로는 `git pull`

| 저장소 | 주소 | 받는 자리 (WSL) |
|---|---|---|
| 강의자료 — 주차 문서 · Simulink 모델 · MATLAB 스크립트 | **https://github.com/wkyouncnu/Capstone-Design** | `~/Capstone-Design` |
| ROS 2 예제 패키지 `usv_basics` — 2주차부터 쓰는 노드 코드 | **https://github.com/wkyouncnu/usv_basics** | `~/capstone_ws/src/usv_basics` |

| 언제 | 명령 (VS Code 의 WSL 창 터미널) | 하는 일 |
|---|---|---|
| 처음 한 번 | `cd ~ && git clone https://github.com/wkyouncnu/Capstone-Design.git` | 강의자료 전체를 받음 (약 770 MB · 2026-10-16 실측 1분 40초) |
| 처음 한 번 | `mkdir -p ~/capstone_ws/src && cd ~/capstone_ws/src && git clone https://github.com/wkyouncnu/usv_basics.git` | 예제 패키지를 받음. 이어서 `cd ~/capstone_ws && colcon build --symlink-install` |
| 매주 수업 전 | `cd ~/Capstone-Design && git pull` · `cd ~/capstone_ws/src/usv_basics && git pull` | **바뀐 파일만** 받음. 다시 clone 하지 않음 |
| 무엇이 바뀌었는지 | `git log --oneline -10` · `git show --stat HEAD` | 교수가 갱신한 내역을 확인 |

- 두 저장소 모두 공개 — 로그인 없이 받아짐
- 주차 문서는 `~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/` 에 있음. MATLAB 은 같은 폴더를 `\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\...` 로 엶
- `usv_basics` 를 받은 뒤 새 노드가 생겼으면 `colcon build --symlink-install` 을 한 번 더 돌림
- 본인이 고친 파일 때문에 `git pull` 이 멈추면 `git stash` 로 치워 두고 다시 받음 → 강의자료는-한-번-받고-git-pull-로-갱신한다

- **과목**: 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- **이번 부록 학습 내용**: 위성지도 위에서 **웨이포인트를 클릭으로 찍고**, 그 위경도를 배가 쓰는 **NED 좌표**로 내림
- **구성**: 1부 이론 (좌표계) · 2부 실습 1~3단

### ★ 시작 전 확인

- **강의자료부터 갱신**: VS Code WSL 창 터미널에서 `cd ~/Capstone-Design && git pull`
- MATLAB **R2024b** 와 **Navigation Toolbox** 가 설치되어 있을 것 ( `lla2ned` 가 여기 있음)
- **인터넷 연결 필요** — 위성 사진은 실시간으로 받아 그림
- 5주차 LOS 유도 실습( `W05_simulink` )을 먼저 해 두면 3단 결과를 바로 이어 쓸 수 있음

### ℹ 이번 부록에서는 ROS 도 Gazebo 도 VRX 도 쓰지 않는다

- WSL 도, 배도 필요 없음. **MATLAB 하나만** 켜면 됨
- 학습 대상은 **위경도와 NED 사이의 변환**, 그리고 **지도를 입력 장치로 쓰는 법**임
- Simulink 연동과 실시간 항적 표시는 **다음 단계(4~7단)** 에서 다룸

## 이 부록을 마치면 할 수 있어야 하는 것

1. 위경도를 거리처럼 쓰면 안 되는 이유를 축척 수치로 설명한다
2. 기준점 `lla0` 하나를 정하고, 그 기준점의 NED 좌표계가 무엇인지 말한다
3. `lla2ned` · `ned2lla` 로 위경도와 NED 를 서로 바꾼다
4. `'flat'` 과 `'ellipsoid'` 중 무엇을 쓸지 **거리를 근거로** 고른다
5. MATLAB 지리축( `geoaxes` )에 위성지도를 띄우고 원하는 범위를 지정한다
6. 지도를 클릭해 웨이포인트를 만들고, W05 모델이 쓰는 `wp_north` · `wp_east` 로 변환한다
7. 클릭한 웨이포인트로 Simulink 모델을 **실시간으로** 돌리고, 배와 항적을 위성지도 위에서 본다
8. 주행 결과를 횡방향 오차·헤딩 오차·최근접 거리의 **수치로** 평가한다

## 준비물

| 항목 | 내용 |
|---|---|
| MATLAB | R2024b 이상 |
| 툴박스 | Navigation Toolbox ( `lla2ned` , `ned2lla` ) |
| 툴박스 | Simulink (4단 실시간 주행) |
| 인터넷 | 위성 베이스맵 타일 수신용 |
| 실습 폴더 | `10-주차별-강의자료/A2_gcs/` |
| 선수 자료 | 5주차 `W05_simulink/` — 4단이 이 모델을 그대로 가져다 씀 |

> **ℹ 지리축 `geoaxes` 자체는 기본 MATLAB 기능임**
>
> - 별도 툴박스 없이 쓸 수 있음
> - 툴박스가 필요한 것은 **좌표 변환** `lla2ned` 쪽임

---

# 1부 · 이론

## 1-1. 위경도를 그대로 쓰면 안 되는 이유

- GPS 가 주는 값은 **각도**임 — 위도(latitude), 경도(longitude), 단위는 도(degree)
- 배의 제어기가 쓰는 값은 **거리**임 — 북쪽으로 몇 m, 동쪽으로 몇 m
- 각도 1도가 몇 m 인지는 **위도마다, 방향마다 다름**

### 용어 정리

| 용어 | 뜻 | 비유 |
|---|---|---|
| 위도 (latitude) | 적도에서 북/남으로 얼마나 올라갔는가 | 지구를 가로로 자른 선의 번호 |
| 경도 (longitude) | 본초자오선에서 동/서로 얼마나 돌았는가 | 지구를 세로로 자른 선의 번호 |
| 고도 (altitude) | 타원체면에서 위로 얼마나 떠 있는가 | 해발 높이와 비슷하나 기준면이 다름 |
| NED | North-East-Down. 북·동·아래 세 방향의 **거리** 좌표 | 운동장 한 귀퉁이를 원점으로 삼은 줄자 |
| 기준점 `lla0` | NED 좌표계의 원점이 될 한 점의 위경도 | 줄자의 0 눈금을 어디에 놓을지 |

- 본 부록의 기준점에서 실제로 잰 축척

| 항목 | 값 |
|---|---|
| 위도 $1 \times 10^{-5}$ ° | 북쪽으로 **1.1097 m** |
| 경도 $1 \times 10^{-5}$ ° | 동쪽으로 **0.8974 m** |
| 위도 1 ° | 110 965.9 m |
| 경도 1 ° | 89 738.4 m |

- **같은 1도인데 북쪽 거리와 동쪽 거리가 다름** — 위도가 높아질수록 경도선 사이가 좁아지기 때문임
- 결론: 위경도 숫자를 빼서 거리로 쓰면 틀림. 반드시 변환해야 함

## 1-2. 기준점과 NED 좌표계

- NED 는 **기준점 하나를 정해야** 성립하는 좌표계임
- 정의

| 축 | 방향 | 기호 |
|---|---|---|
| $x$ | 북(North) | `xNorth` |
| $y$ | 동(East) | `yEast` |
| $z$ | **아래(Down)** | `zDown` |

> ⚠ $z$ 축이 아래를 향한다
>
> - 수면 **위**에 있으면 $z$ 는 **음수**임
> - 고도 10 m 상공 = `zDown = -10`
> - 부호를 반대로 쓰면 오류 메시지 없이 결과만 뒤집힘

- 본 부록의 기준점

| 항목 | 값 |
|---|---|
| 위치 | 갑천 자전거길, 대전광역시 |
| 위도 | 36.3719625 ° N |
| 경도 | 127.3927656 ° E |
| 고도 `alt0` | 0 m |

> ℹ **기준점 고도를 0 으로 둔 근거**
> - `'flat'` 방식에서 `alt0` 을 0 과 50 m 로 바꿔 실측한 결과, 수평 좌표(N, E)의 차는 **정확히 0** 이었음
> - 바뀌는 것은 D 성분뿐임
> - 즉 `alt0 = 0` 은 추측이 아니라 **영향이 없음을 확인한 선택**임

## 1-3. `lla2ned` 와 `ned2lla`

- MATLAB 이 제공하는 변환 함수 한 쌍

```
xyzNED = lla2ned(lla, lla0, method)
lla    = ned2lla(xyzNED, lla0, method)
```

| 인자 | 내용 |
|---|---|
| `lla` | `[위도 경도 고도]` — 도, 도, m |
| `xyzNED` | `[xNorth yEast zDown]` — 전부 m |
| `lla0` | 기준점. `1×3` |
| `method` | `'flat'` 또는 `'ellipsoid'` |

> ⚠ **세 번째 인자는 생략할 수 없다**
> - 함수 내부가 `narginchk(3,3)` 이라 기본값이 없음
> - `lla2ned(p, lla0)` 로 부르면 오류가 남

- 여러 점을 한 번에 변환 가능 — `lla` 가 `N×3` 이면 결과도 `N×3`
- 반복문을 쓰지 않는 것이 더 빠르고 짧음

## 1-4. `'flat'` 과 `'ellipsoid'` 중 무엇을 쓰는가

| 방식 | 가정 |
|---|---|
| `'flat'` | 지구를 평면으로 보고, $z$ 축이 **기준점에서만** 지표에 수직 |
| `'ellipsoid'` | WGS84 타원체를 그대로 사용 |

- 두 방식의 차를 거리를 늘려 가며 실측한 결과 (성분 분리)

| 원점에서 거리 | 수평 차 | 상대오차 | 수직 항 |
|---|---|---|---|
| 100 m | 0.0000 m | 0.1 ppm | 0.0008 m |
| 300 m | 0.0001 m | 0.2 ppm | 0.0071 m |
| 500 m | 0.0002 m | 0.4 ppm | 0.0197 m |
| 1 km | 0.0008 m | 0.8 ppm | 0.0786 m |
| 2 km | 0.0031 m | 1.5 ppm | 0.3146 m |
| 5 km | 0.0199 m | 4.0 ppm | 1.9661 m |

- 두 성분의 성격이 완전히 다름

| 성분 | 크기 | 원인 |
|---|---|---|
| 수평 차 | 5 km 에서도 **2 cm** | 평면 근사의 실제 오차 |
| 수직 항 | 5 km 에서 **1.97 m** | 지구 곡률 $L^2/(2R)$ |

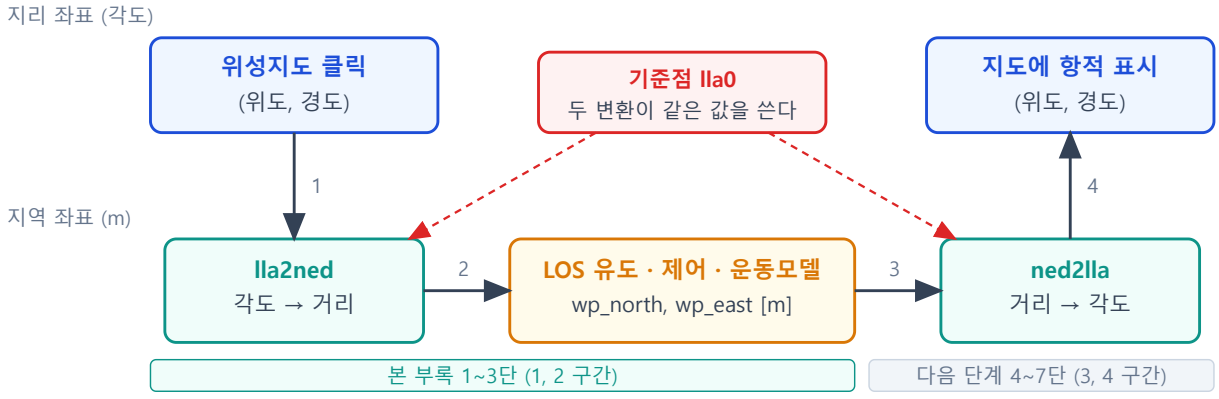
> **수직 항이 왜 생기는가**
>
> - `'ellipsoid'` 의 NED 는 기준점에 붙인 **접평면 직교좌표**임
> - 수평으로 $L$ 만큼 간 점은 타원체면에서 아래로 휘어 내려가므로, 접평면에서 보면 $L^2/(2R)$ 만큼 **떠오른** 점이 됨
> - $R = 6\,371$ km 를 넣으면 $5000^2/(2 \times 6.371 \times 10^6) = 1.96$ m — 실측값 1.9661 m 와 일치함

$$\Delta z \approx \frac{L^2}{2R}$$

- **본 과목의 선택:** `'flat'`
  - 코스 규모가 수백 m 이므로 수평 오차는 1 mm 미만임
  - 배가 수면 위에서만 움직이므로 수직 항은 쓰지 않음
  - 참조 구현(지난 학기 VRX 스크립트)도 `'flat'` 만 사용함
- 수 km 이상으로 넓히면 **방식을 다시 고를 것**

## 1-5. 전체 흐름

**지도 클릭에서 배 표시까지 — 좌표가 지나가는 길**



| 번호 | 단계 | 쓰는 함수 |
|---|---|---|
| 1 | 지도를 클릭해 위경도를 얻음 | `ginput` |
| 2 | 위경도를 NED 로 내림 | `lla2ned` |
| 3 | NED 웨이포인트를 유도·제어기에 넣음 | W05 모델 |
| 4 | 배의 NED 위치를 다시 위경도로 올림 | `ned2lla` |
| 5 | 지도 위에 항적을 그림 | `geoplot` |

- 1~3 이 본 문서의 범위임. 4~5 는 다음 단계임

---

# 2부 · 실습

## 2-1. 파일 구성

- 실습 폴더: `10-주차별-강의자료/A2_gcs/`

| 파일 | 무엇 |
|---|---|
| `A2_setup.m` | **기준점과 설정 한 곳** — 다른 모든 스크립트가 맨 처음 부름 |
| `A2_openmap.m` | 지도 축을 만들어 돌려주는 도우미 함수 |
| `A2_1_refpoint_todo.m` / `_done.m` | 1단 — 숫자만. 지도 없음 |
| `A2_2_map_todo.m` / `_done.m` | 2단 — 위성지도와 기준점 |
| `A2_3_click_todo.m` / `_done.m` | 3단 — 클릭으로 웨이포인트 만들기 |
| `A2_4_live_todo.m` / `_done.m` | 4단 — 클릭한 웨이포인트로 실시간 주행 |
| `A2_live_setup.m` | 4단 설정 — 5주차 제어 설정 위에 기준점과 웨이포인트를 얹음 |
| `A2_build_live_model.m` | 4단 모델 생성 — `W05_0_offline.slx` → `A2_live.slx` |
| `A2_animate_map.m` | 4단 화면 — 위성지도 위의 배·항적과 오른쪽 그래프 |
| `mission_points.mat` | 3단이 저장하는 웨이포인트 — 예시 한 벌이 함께 배포됨. 실행하면 덮어씀 |
| `A2_live.slx` | 4단이 자동 생성하는 모델. 손으로 고치지 않음 |

> ★ `_todo` 를 먼저 풀고, 막히면 `_done` 을 연다
> - `_todo` 파일에는 빈칸(TODO)이 있고, 채우지 않고 돌리면 **어느 TODO 가 비었는지 알려 주고 멈춤**
> - `_done` 을 먼저 열면 배우는 것이 없음

- MATLAB 현재 폴더를 `A2_gcs` 로 옮기고 시작함

```
cd 10-주차별-강의자료/A2_gcs
```

## 2-2. 1단 — 기준점과 좌표 변환 (지도 없음)

- 지도를 띄우기 전에 **숫자만** 본다. 지도가 끼면 무엇이 틀렸는지 알기 어려움

```
A2_1_refpoint_done
```

- 정상 출력

```
=== 1단 · 기준점과 좌표 변환 ===

기준점 lla0 = [36.3719625  127.3927656  0.0]

[1] 축척
  1e-5 deg  = 북 1.1097 m / 동 0.8974 m
  위도 1 deg = 110965.9 m
  경도 1 deg = 89738.4 m
  -> 위도와 경도의 m 환산값이 다르다. 각도를 거리처럼 쓰면 안 되는 이유다

[2] 왕복
  넣은 NED   : N  300.000  E  200.000  D    0.000
  위경도     : 36.3746660 , 127.3949943
  돌아온 NED : N  300.000  E  200.000  D    0.000
  왕복 오차  : 3.335e-10 m
```

- 확인할 것

| 항목 | 기준 |
|---|---|
| 왕복 오차 | `1e-9 m` 미만이면 정상 — 변환 자체는 손실이 없음 |
| 북쪽 300 m 가 위도로 | 약 `+0.0027 °` |
| 동쪽 200 m 가 경도로 | 약 `+0.0022 °` |

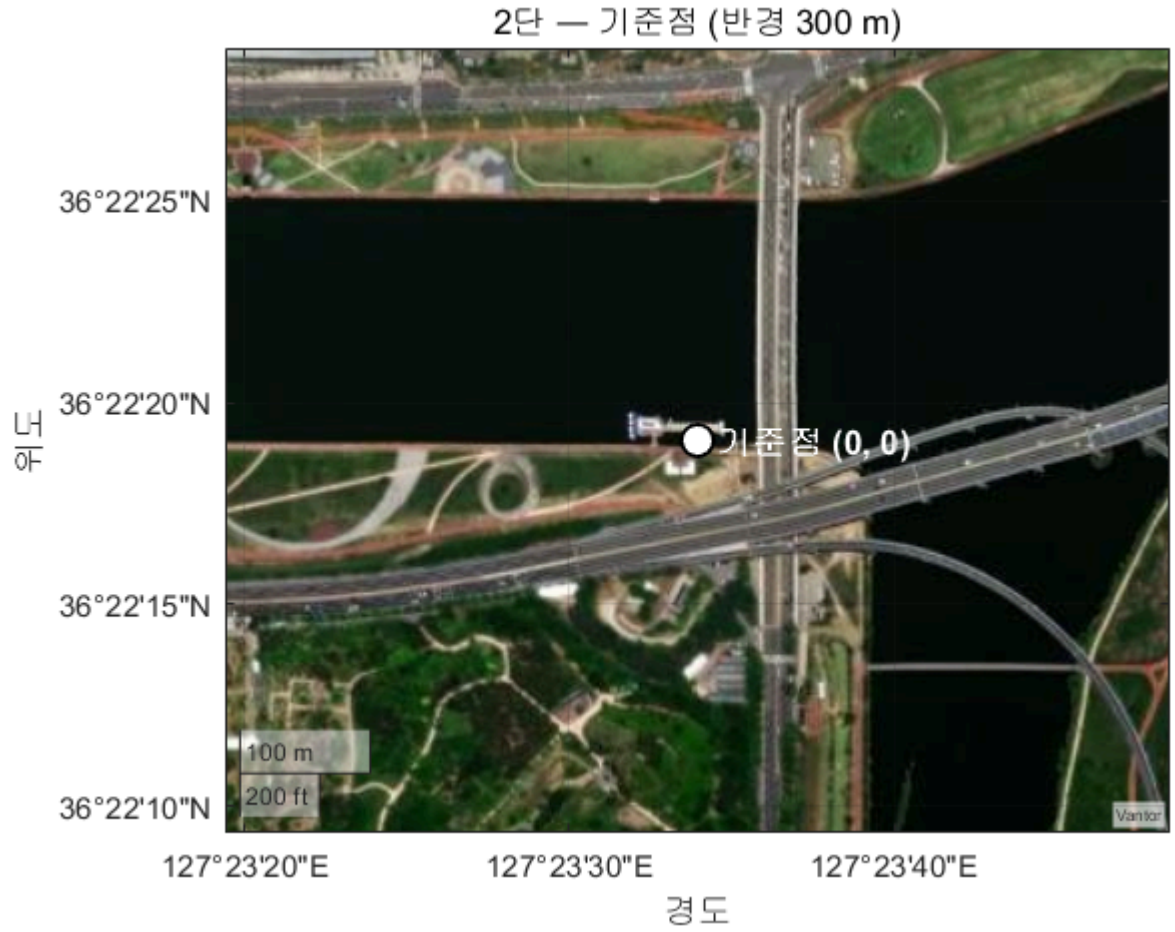
- 이어지는 `[3]` 평면 가정의 오차, `[4]` 기준점 고도의 영향은 1부 1-4 와 같은 숫자가 나옴

> ✎ **왕복 오차가 왜 0 이 아닌가**
> - 부동소수점 계산의 반올림일 뿐임
> - `3.335e-10 m` = 0.3 나노미터. 원자 크기보다 작음

## 2-3. 2단 — 위성지도를 띄우고 기준점 찍기

```
A2_2_map_done
```

- 화면에 보이는 것




| 보이는 것 | 뜻 |
|---|---|
| 위성 사진 | 인터넷에서 받아 온 베이스맵 타일 |
| 가운데 흰 점 | 기준점 — NED 좌표로 `(0, 0)` |
| 축 눈금 | 도·분·초 표기의 위도·경도 |
| 왼쪽 아래 축척 막대 | 100 m / 200 ft |

- 정상 출력

```
[1] 지도 범위
  요청 위도 : [36.369259  36.374666]
  실제 위도 : [36.369259  36.374666]
  요청 경도 : [127.389423  127.396109]
  실제 경도 : [127.388728  127.396804]

[2] 실제 범위를 m 로
  남서 꼭짓점 : N   -300.0  E   -362.4
  북동 꼭짓점 : N    300.0  E    362.4
  지도 크기   : 북 600.0 m x 동 724.7 m
```

**⚠ 요청한 범위와 실제 범위가 다르다**

- `geolimits` 는 **창의 가로세로 비율을 지키려고** 요청값보다 넓게 잡음
- 위 출력에서 위도는 그대로인데 경도만 넓어졌음 (요청 `±300 m` → 실제 `±362.4 m`)
- 검증할 때는 반드시 **되읽은 값**을 쓸 것

```
[LA, LO] = geolimits(gx);      % 실제 적용된 범위
```

- 지도 범위를 m 로 지정하는 요령 — 각도를 손으로 나누지 않음

```
sw = ned2lla([-span_m, -span_m, 0], lla0, 'flat');
ne = ned2lla([ span_m,  span_m, 0], lla0, 'flat');
geolimits(gx, [sw(1) ne(1)], [sw(2) ne(2)]);
```

- 위도와 경도의 m 환산이 다르므로(1-1), **같은 각도를 더하면 정사각형이 되지 않음**. 변환 함수에 맡김

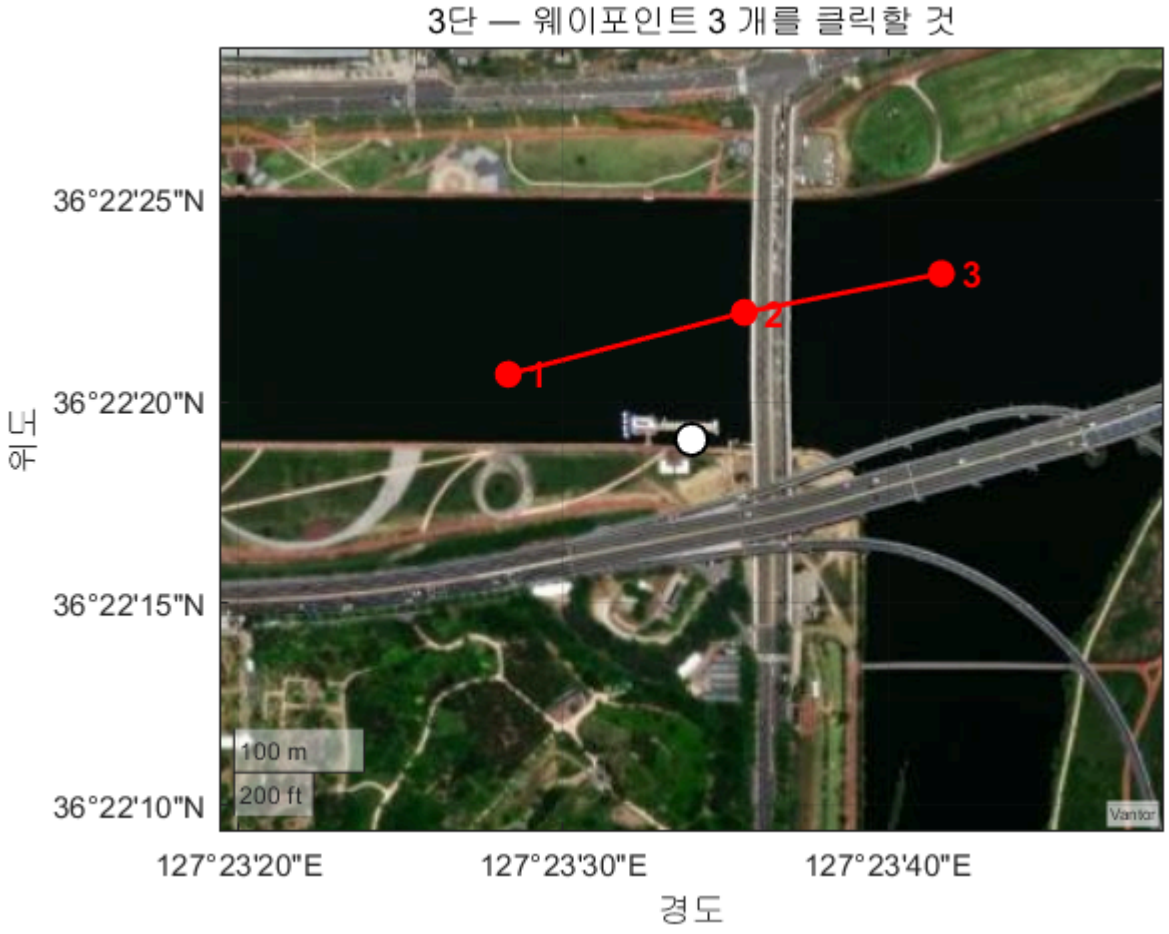
**✕ 인터넷이 없으면 위성 사진이 나오지 않는다**

- `'satellite'` 베이스맵은 **미리 받아 둘 수 없음** — 실행할 때마다 타일을 내려받음
- 인터넷이 없으면 `A2_openmap` 이 경고를 내고 `'darkwater'` (MATLAB 에 들어 있는 기본 지도)로 자동 대체함
- 위성 사진은 안 보이지만 **좌표 계산과 클릭은 그대로 동작함**

## 2-4. 3단 — 지도를 클릭해 웨이포인트 만들기

```
A2_3_click_done
```

- 실행하면 지도가 뜨고 **MATLAB 이 멈춘 것처럼 보임** — 클릭을 기다리는 중임
- 지도 위를 세 번 클릭하면 빨간 점과 번호가 찍히고 경로가 이어짐



### ★ `ginput` 은 위도가 먼저 돌아온다

```
[lat, lon] = ginput(1);     % 첫째가 위도, 둘째가 경도
```

- 보통의 축에서는 `[x, y]` 지만 **지리축에서는** `[위도, 경도]` 임
- 순서를 바꿔 받으면 오류 메시지 없이 엉뚱한 자리에 점이 찍힘

- 정상 출력 (클릭 위치에 따라 숫자는 달라짐)

```
지도 위에서 웨이포인트 3 개를 차례로 클릭할 것
  (클릭할 때까지 MATLAB 은 기다린다 — ginput 은 블로킹이다)

  1 번째 클릭 : 36.3724118 , 127.3911974
  2 번째 클릭 : 36.3728374 , 127.3932165
  3 번째 클릭 : 36.3731054 , 127.3949023

저장함 : ...\A2_gcs\mission_points.mat

[웨이포인트]
    #            위도             경도         N[m]         E[m]
    1    36.3724118    127.3911974       49.86     -140.73
    2    36.3728374    127.3932165       97.09       40.46
    3    36.3731054    127.3949023      126.83      191.74

[구간 거리]
  1 -> 2 :  187.24 m
  2 -> 3 :  154.18 m
  기준점 -> 1 :  149.30 m
```

- 읽는 법

| 열 | 뜻 |
|---|---|
| 위도·경도 | 클릭한 자리의 지리 좌표 |
| N | 기준점에서 **북쪽으로** 몇 m — 음수면 남쪽 |
| E | 기준점에서 **동쪽으로** 몇 m — 음수면 서쪽 |

- 위 예에서 1번 점의 E 가 `-140.73` 이므로 기준점보다 **서쪽**에 있음. 지도에서도 왼쪽에 찍혀 있음

## 변환은 반복문 없이 한 줄로 한다

```
wp_ned = lla2ned(wp_lla, lla0, 'flat');   % wp_lla 는 N x 3
```

- `lla2ned` 는 `N×3` 입력을 그대로 받아 `N×3` 을 돌려줌
- 점이 100 개여도 같은 한 줄임

## W05 모델이 쓰는 이름으로 맞춘다

```
wp_north = wp_ned(:,1).';
wp_east  = wp_ned(:,2).';
```

- 5주차 `W05_setup.m` 이 쓰는 변수 이름과 **글자 하나까지 같음**
- 따라서 이 두 줄의 결과를 그대로 넣으면 LOS 유도 모델이 돈다

> ✎ **한 번 찍은 점을 다시 쓴다**
>
> - 매번 새로 클릭하면 실험 조건이 달라져 결과를 비교할 수 없음
> - 스크립트 머리의 `USE_SAVED` 를 `true` 로 바꾸면 `mission_points.mat` 을 다시 읽음
>
> ```
> USE_SAVED = true;
> ```

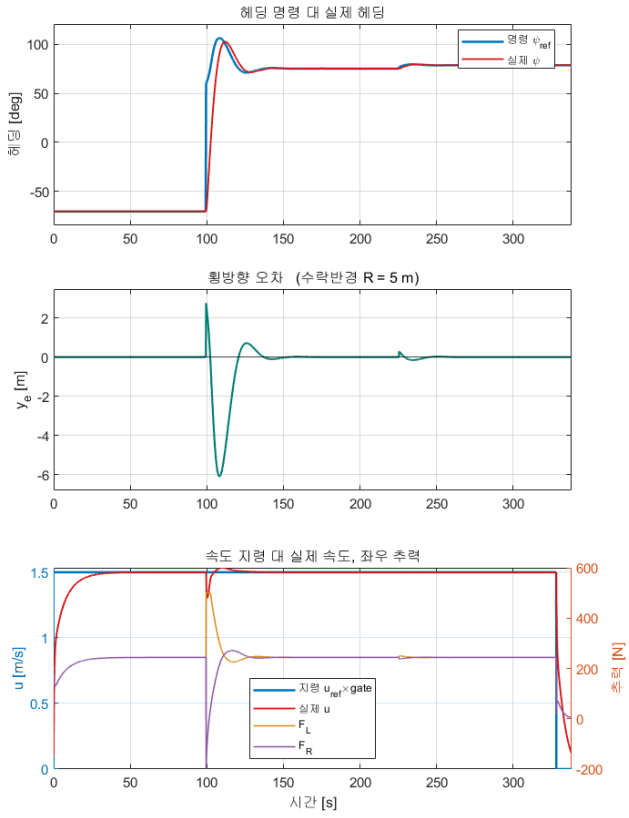
## 2-5. 4단 — 클릭한 웨이포인트로 실시간 주행

> ★ **이 단이 부록 A2 의 목적지임**
>
> - 지도에서 찍은 점이 **그대로** Simulink 유도부의 웨이포인트가 됨
> - 모델이 도는 동안 배와 항적이 **위성지도 위에** 실시간으로 그려짐
> - 헤딩 명령과 실제 헤딩을 **같은 화면에서 나란히** 봄

```
A2_4_live_done
```

- 3단에서 저장한 `mission_points.mat` 을 읽으므로 **3단을 먼저 돌려야 함**
- 500 m 경로를 실시간으로 도므로 **약 5분 30초** 걸림. 중간에 창을 닫지 않음



## 화면 읽는 법

| 위치 | 색 | 무엇 |
|---|---|---|
| 지도 | 흰 선 | 계획 경로 — 클릭한 점을 이은 것 |
| 지도 | 빨간 점 + 번호 | 웨이포인트. `0` 은 출발점 |
| 지도 | 노란 선 | 지나온 항적 |
| 지도 | 청록 선 | 선체 — 뱃머리가 향한 쪽이 보임 |
| 지도 | 빨간 짧은 선 | 현재 헤딩 방향 |
| 오른쪽 위 | 파랑 / 빨강 | 헤딩 **명령** psi_ref / **실제** 헤딩 psi |
| 오른쪽 가운데 | 청록 | 횡방향 오차 y_e — 계획 경로에서 옆으로 벗어난 거리 |
| 오른쪽 아래 | 4색 | 속도 지령·실제 속도·좌우 추력 |

- 지도 제목에 현재 시각·구간 번호·횡방향 오차·속도가 함께 뜸

## 파일 구성

| 파일 | 무엇 |
|---|---|
| `A2_live_setup.m` | 5주차 제어 설정 + 기준점 + 클릭한 웨이포인트를 한 번에 깖 |
| `A2_build_live_model.m` | 5주차 오프라인 모델에서 `A2_live.slx` 를 **자동 생성** |
| `A2_animate_map.m` | 위성지도 위에 배·항적·그래프를 그리는 함수 |
| `A2_4_live_todo.m` / `_done.m` | 4단 실행 스크립트 |
| `A2_live.slx` | 생성된 모델 — **손으로 고치지 않음.** 지우면 다시 만들어짐 |

> ★ **모델을 새로 그리지 않고 5주차 모델을 그대로 가져옴**
>
> - `A2_build_live_model` 이 `W05_0_offline.slx` 를 복사해 그림 함수 이름 하나만 바꿈
> - 제어기·추진기·운동모델·이득은 **5주차에서 검증한 것과 같음**
> - 따라서 결과가 달라졌다면 원인은 **웨이포인트뿐**임 — 비교가 가능해짐
> - 5주차 원본 파일은 건드리지 않음

## 출발점을 0번 웨이포인트로 넣는 이유

- 5주차 유도부는 `wp(k)` 에서 `wp(k+1)` 로 가는 구간을 따라감
- 클릭한 점만 넣으면 1번 점이 **목표가 아니라 첫 구간의 시작점**이 되어 버림
- `A2_live_setup` 이 맨 앞에 `(0, 0)` (= 기준점)을 끼워 넣음

```
wp_north = [0, wp_clicked(:,1).'];
wp_east  = [0, wp_clicked(:,2).'];
```

- 결과: 클릭 3개 + 출발점 1개 = **경로 점 4개, 구간 3개**

## 실시간으로 돌리는 한 줄

```
set_param(m, 'EnablePacing', 'on', 'PacingRate', '1');
```

- 페이싱을 끄면 수백 배 빨리 돌아 **화면을 볼 수 없음**
- `PacingRate` 는 **문자열**로 줌

- 요청한 배율이 그대로 나오지는 않음 — 실행 끝에 **실효 배율**을 출력해 확인함

> **⚠ 배율을 높인다고 그만큼 빨라지지 않음**
>
> - 요청 10 배로 돌렸을 때 실효 **5.26 배**에 그침 (기준 환경 실측)
> - 그림 그리기가 따라오지 못하면 모델이 기다림
> - 결과만 빨리 보고 싶을 때만 올리고, 제출용 측정은 **1 배**로 함

## 정상 출력

```
=== 4단 · 실시간 주행 ===
페이싱 1.0 배 — 예상 소요 시간 327 s

[측정]
  실행 시간          : 시뮬 338.2 s / 벽시계 343.9 s  -> 실효 0.983 배
  완주 시각          : 328.2 s  (최소 소요 327.1 s 의 1.00 배)
  평균 |y_e|         : 전 구간 0.238 m / 초기 20 s 제외 0.253 m
  최대 |y_e|         : 6.095 m
  헤딩 오차 RMS      : 전 구간 12.20 deg / 초기 20 s 제외 12.59 deg
  최대 |헤딩 오차|   : 130.59 deg
  평균 속도          : 1.499 m/s  (지령 1.50 m/s)

[웨이포인트 최근접 거리]
    #        최근접[m]         그때[s]
    0          0.00            0.0
    1          1.42          102.4
    2          0.03          228.7
    3          4.99          328.2
  수락반경 R = 5 m
```

## 측정값 읽기 — 숫자가 말하는 것

| 숫자 | 읽는 법 |
|---|---|
| 실효 **0.983 배** | 요청 1 배를 거의 그대로 따라감. 실시간으로 본 것이 맞음 |
| 완주 **328.2 s** | 최소 소요 327.1 s 의 **1.00 배** — 멀리 돌아간 구간이 없음 |
| 평균 횡방향 오차 **0.238 m** | 전 구간 평균. 계획 경로에서 평균 24 cm 안쪽에 붙어 감 |
| 최대 횡방향 오차 **6.095 m** | 수락반경 5 m 보다 큼 — **한 지점에서만** 벌어진 값임 |
| 헤딩 오차 RMS **12.20 deg** | 직선 구간이 아니라 **선회 한 번**이 올린 값 |
| 최대 헤딩 오차 **130.59 deg** | 명령이 계단처럼 꺾인 순간. 배가 즉시 돌 수는 없음 |
| 최근접 **1.42 / 0.03 / 4.99 m** | 세 점 모두 수락반경 5 m 안으로 들어감 = 전 구간 완주 |

**ℹ 최대 오차가 한 지점에만 몰리는 이유**

- 1번 웨이포인트에서 경로가 **145.88 °** 꺾임 (구간 방위 −70.49 ° → 75.39 °)
- 2번에서는 **3.49 °** 만 꺾임
- 즉 급선회는 전 구간에 한 번뿐이며, 최대 횡방향 오차 6.095 m 와 최대 헤딩 오차 130.59 deg 는 **모두 이 한 번에서 나온 값**임
- 지도의 1번 점 주변에 **고리 모양 항적**이 보이는 것이 그 흔적임

**⚠ 오프라인 모델에는 육지가 없다**

- 본 단계의 운동모델은 **지면·수심·장애물을 모르는** 평면 모델임
- 지도에서 둑이나 다리 위를 지나는 경로를 찍어도 그대로 지나감
- 지도 위에 그려진다고 해서 그 경로가 **항행 가능한 경로는 아님**
- 실제 수역 판정은 VRX 연동 단계에서 다룸

## 측정은 구간을 밝혀서 쓴다

- 평균 횡방향 오차를 **전 구간**과 **초기 20 s 제외** 두 가지로 냄
- 이유: 출발 직후에는 정지 상태에서 속도를 올리는 과도구간이 섞임
- 과제 제출 시 **어느 구간의 값인지 반드시 적을 것** → 측정하지-않은-성공은-성공이-아니다

---

# 마무리

## 이번 부록 요약

| 항목 | 내용 |
|---|---|
| 기준점 | 36.3719625 N, 127.3927656 E, 고도 0 (갑천) |
| 축척 | 위도 1 ° = 110 965.9 m · 경도 1 ° = 89 738.4 m |
| 변환 함수 | `lla2ned` · `ned2lla`, 세 번째 인자 필수 |
| 선택한 방식 | `'flat'` — 수백 m 코스에서 수평 오차 1 mm 미만 |
| 왕복 오차 | `3.335e-10 m` |
| 지도 | `geoaxes` + `geobasemap('satellite')`, 기본 MATLAB 기능 |
| 범위 지정 | `geolimits` 는 종횡비 때문에 넓어짐 — 되읽을 것 |
| 클릭 입력 | `ginput(1)` → **(위도, 경도)** 순 |
| 결과 | `wp_north` · `wp_east` — W05 모델에 그대로 들어감 |
| 4단 모델 | `W05_0_offline.slx` 를 복사해 그림 함수만 교체 — 제어는 5주차와 동일 |
| 실시간 | `EnablePacing` + `PacingRate` (문자열). 요청 1 배 → 실효 **0.983 배** |
| 4단 측정 | 완주 328.2 s · 평균 횡방향 오차 0.238 m · 헤딩 오차 RMS 12.20 deg |
| 4단 최근접 | 1.42 / 0.03 / 4.99 m — 세 점 모두 수락반경 5 m 안 |

## 수업 진도 체크

> ★ 다음 단계를 따라가기 위한 최소 조건

### 이론 이해

- [ ] 위도 1 °와 경도 1 °의 m 환산이 왜 다른지 설명할 수 있다
- [ ] NED 의 $z$ 축이 아래를 향한다는 것과, 수면 위가 음수라는 것을 말할 수 있다
- [ ] `'flat'` 을 고른 근거를 **거리와 오차 수치로** 댈 수 있다
- [ ] 출발점을 0번 웨이포인트로 끼워 넣는 이유를 설명할 수 있다
- [ ] 페이싱을 켜지 않으면 화면을 볼 수 없는 이유를 설명할 수 있다

### 실습 완료

- [ ] `A2_1_refpoint_todo` 의 TODO 세 개를 채워 `_done` 과 같은 출력을 냈다
- [ ] `A2_2_map_todo` 를 채워 위성지도에 기준점을 찍었다
- [ ] `A2_3_click_todo` 를 채워 웨이포인트 3개를 클릭으로 만들었다
- [ ] `mission_points.mat` 의 저장 시각이 갱신됐다
- [ ] `A2_4_live_todo` 의 TODO 세 개를 채워 실시간 주행을 끝까지 돌렸다
- [ ] 위성지도 위에서 배가 계획 경로를 따라가는 것을 **눈으로** 확인했다

### 관찰 기록

- [ ] 2단에서 요청한 지도 범위와 실제 범위의 차를 숫자로 적었다

- [ ] 3단에서 만든 웨이포인트의 N·E 값과 구간 거리를 적었다
- [ ] 4단의 실효 페이싱 배율을 적었다 (요청값과 함께)
- [ ] 4단의 평균·최대 횡방향 오차와 헤딩 오차 RMS 를 **구간을 밝혀** 적었다
- [ ] 각 웨이포인트 최근접 거리를 수락반경 5 m 와 비교해 적었다

### 신고 사항

- [ ] 위성 사진이 뜨지 않았다면(인터넷 문제) 그 사실을 적었다

## 과제 A2

> **★ 제출물: MATLAB 스크립트 1개 + 결과 그림 3장 + 표 2개**

### 요구사항

1. 기준점 `lla0` 을 **본인이 고른 다른 장소**로 바꿀 것 (Google Maps 에서 좌표를 읽어 옴)
2. 그 기준점에서 1단을 다시 돌려 **축척 표**를 만들 것 (위도·경도 1 ° 의 m 환산)
3. 위성지도를 띄우고 웨이포인트를 **5개** 클릭해 폐곡선 경로를 만들 것
4. 웨이포인트 표(위도·경도·N·E)와 **구간 거리 합계**를 낼 것
5. 같은 웨이포인트를 `'flat'` 과 `'ellipsoid'` 두 방식으로 변환해 **차이를 m 로** 제출할 것
6. 그 경로로 4단을 **페이싱 1 배**로 끝까지 돌리고, 완주 직후 화면을 그림으로 낼 것
7. 주행 측정값을 표로 낼 것 — 완주 시각 · 평균/최대 횡방향 오차 · 헤딩 오차 RMS · 각 웨이포인트 최근접 거리
8. 최대 횡방향 오차가 **어느 웨이포인트에서** 나왔는지 밝히고, 그 지점의 **선회각(deg)** 과 함께 설명할 것

### 평가 배점

| 항목 | 배점 | 기준 |
|---|---|---|
| 기준점 변경과 축척 표 | 15 | 본인 기준점의 위도·경도 1 ° 환산값이 맞는가 |
| 지도 표시 | 10 | 기준점과 5개 웨이포인트가 지도 위에 보이는가 |
| NED 변환 표 | 10 | 위경도 → N·E 변환이 맞는가 |
| **검증 — 왕복 오차** | **15** | `lla2ned` → `ned2lla` 왕복 오차를 **수치로** 제출했는가 |
| **검증 — 두 방식 비교** | **15** | `'flat'` 과 `'ellipsoid'` 의 차를 **수평·수직 성분으로 나누어** 제출했는가 |
| 4단 실시간 주행 | 15 | 본인 경로로 완주했고 화면에 항적이 남았는가 |
| **검증 — 주행 측정값** | **15** | 측정값을 **구간을 밝혀** 표로 냈고, 최대 오차 지점을 선회각으로 설명했는가 |
| 코드 가독성 | 5 | 반복문 없이 `N×3` 일괄 변환을 썼는가 |

- 검증 항목 합계 **45%** — 숫자 없는 "잘 됐습니다" 는 인정하지 않음 → 측정하지-않은-성공은-성공이-아니다

## 막혔을 때

| 증상 | 원인 | 해결 |
|---|---|---|
| `정의되지 않은 함수 'lla2ned'` | Navigation Toolbox 없음 | `ver` 로 설치 목록 확인. 없으면 담당 교수에게 신고 |
| `narginchk` 오류 | 세 번째 인자( `'flat'` )를 빠뜨림 | `lla2ned(p, lla0, 'flat')` 로 세 인자 모두 전달 |
| 지도가 회색·단색으로만 나옴 | 인터넷 없음 → `darkwater` 로 대체됨 | 네트워크 확인. 좌표 계산은 그대로 동작함 |
| 점이 엉뚱한 자리에 찍힘 | `ginput` 반환 순서를 `[lon, lat]` 로 받음 | **첫째가 위도**임 |
| MATLAB 이 멈춘 것 같음 | `ginput` 이 클릭을 기다리는 중 | 지도 창을 클릭. 취소하려면 지도 창을 닫음 |
| `TODO n 을 먼저 채울 것` | `_todo` 의 빈칸을 안 채움 | 해당 TODO 블록을 채움 |
| 지도 범위가 요청과 다름 | 종횡비 보정 | 정상임. `geolimits(gx)` 로 되읽어 사용 |
| 웨이포인트가 매번 달라짐 | 매 실행마다 새로 클릭함 | `USE_SAVED = true` 로 두고 저장본 사용 |
| 4단에서 `mission_points.mat` 없음 | 3단을 아직 안 돌림 | `A2_3_click_done` 을 먼저 실행 |
| 4단 화면이 순식간에 끝남 | 페이싱이 꺼져 있음 | `set_param(m,'EnablePacing','on', ...)` 확인 |
| 실효 배율이 요청값보다 훨씬 낮음 | 그림 그리기가 모델을 붙잡음 | 정상임. 측정용은 1 배로 돌림 |
| 모델이 저장 안 됨 오류 | `A2_live` 가 열린 채 수정돼 있음 | 창을 닫거나 `A2_build_live_model(true)` 로 다시 만듦 |
| 배가 출발점에서 엉뚱한 쪽으로 감 | 0번 웨이포인트가 빠짐 | `A2_live_setup` 을 거치지 않고 직접 변수를 넣었는지 확인 |
| `기준점이 바뀌었다` 경고 | 3단 저장 당시와 `lla0` 이 다름 | 3단을 같은 기준점에서 다시 실행 |

## 참고 자료

- MathWorks `lla2ned` · `ned2lla` 문서 (Navigation Toolbox)
- MathWorks `geoaxes` · `geobasemap` · `geolimits` 문서 (기본 MATLAB)
- 5주차 W05_웨이포인트_유도_atan2와_LOS — `wp_north` · `wp_east` 를 쓰는 모델
- MathWorks `EnablePacing` · `PacingRate` 문서 (Simulink 시뮬레이션 페이싱)
- ENU와-NED를-섞으면-조용히-틀린다

## 다음 단계

**ℹ 5단은 준비 중임**

| 단 | 무엇 | 상태 |
|---|---|---|
| 1~3 | 기준점 · 위성지도 · 클릭 웨이포인트 | 완료 |
| 4 | 클릭한 웨이포인트로 **실시간 주행** — 지도 위의 배·항적·헤딩 | 완료 |
| 5 | **버튼과 표시창을 갖춘 GCS 화면** — 실행·정지·웨이포인트 추가를 화면에서 | 준비 중 |
| 그 다음 | VRX 연동 — 실제 수역·장애물이 있는 시뮬레이터로 같은 임무 | 미정 |

- 4단은 당초 계획의 4~6단(재생 · 실시간 표시 · 모델 연동)을 **한 단으로 합친 것**임
  - 재생을 따로 둘 이유가 없었음 — 실시간 표시가 되면 재생은 같은 코드의 특수한 경우임